CloudWatch AI
Real-Time AIOps Platform for Predictive Incident Detection

Akinbobola Akin Alfarizy Alfarizy Ishimwe Pacis Hanyurwimfura

University of New Brunswick
CS6905 – Cloud Information Management Systems

March 2026

Project Introduction

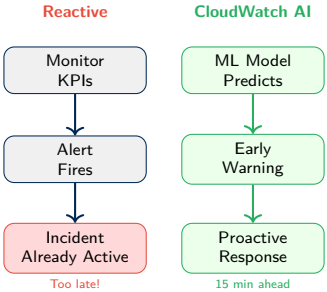
Predicting Cloud Incidents Before They Happen

The Problem

- Cloud systems alert operators **after** services degrade
- Reactive monitoring leads to **slow response times**
- High false-alarm rates cause **alert fatigue**
- Operators lose trust in monitoring systems

Our Solution: CloudWatch AI

A real-time AIOps platform that uses **Gradient Boosting ML models** to predict service incidents **up to 15 minutes in advance**, achieving **0.91 precision** with fewer than 1 false alarm per service per day.



Key Features

What Makes CloudWatch AI Unique

Predictive Intelligence

- 3 ML models for 5, 10, 15-min horizons
- 8 engineered features (KPI + Log signals)
- Calibrated thresholds per service
- Lead time: ∼36 minutes average

Real-Time Simulation

- 5 monitored cloud services
- 6 configurable failure scenarios
- Live data generation every 1 minute
- Event-driven pipeline via DynamoDB Streams

Interactive Dashboard

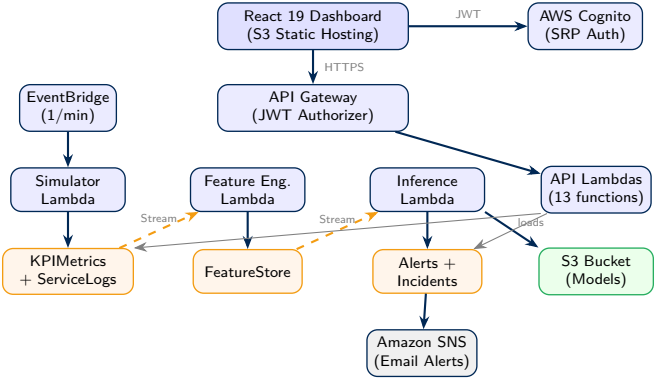
- 4 pages: Live Controls, KPI Timeline, Alerts, Analytics
- Dark/Light theme with persistent preference
- Real-time chart updates via polling
- Alert management with incident grouping
- Opt in/out Incident Notification on Email

Enterprise Security

- AWS Cognito authentication (SRP protocol)
- JWT authorizer on all 19 API routes
- AES-256 encryption at rest
- TLS 1.2+ encryption in transit

System Architecture

Serverless, Event-Driven Architecture on AWS



Key: Solid arrows = direct calls. Dashed orange = DynamoDB Stream triggers (automatic, event-driven).

13 Lambda functions total, all serverless. Entire platform runs within AWS Free Tier.

Technology Stack

AWS Services and Open-Source Tools

Frontend

Technology	Purpose
React 19 + TypeScript	UI framework
Vite 7	Build tool
Tailwind CSS 4	Styling
Recharts 3.8	Charts
Cognito SDK	Authentication
React Router 7	Navigation

Backend & Infrastructure

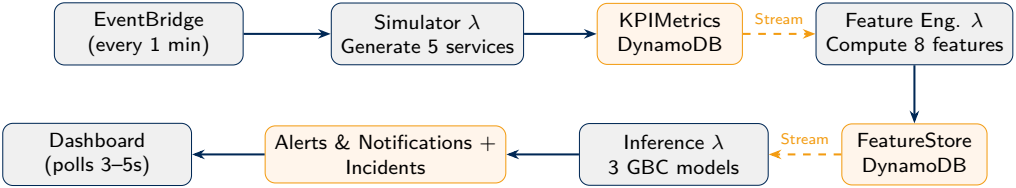
Service	Purpose
AWS Lambda	Compute (Python)
DynamoDB	Database (6 tables)
API Gateway	HTTP API (19 routes)
S3	Static hosting + models
EventBridge	Scheduler (1/min)
Cognito	Authentication
SNS	Email notifications
IAM	Access control & permissions
Docker	Containerization

ML Pipeline

Component	Technology	Details
Model	scikit-learn GBC	100 trees, depth 3, LR 0.1
Training Data	AIOps KPI Dataset	2.4M timesteps, 26 services
Deployment	Docker on Lambda	Image from ECR

Data Flow Pipeline

Event-Driven, Fully Automatic Processing



6 Simulation Scenarios

- **Web API Degradation** – response times spike
- **DB Cascade Failure** – deadlocks cascade
- **MQ Backlog** – queue depth spikes
- **Auth Failure** – login rate drops
- **ML Drift** – accuracy degrades
- **Baseline** – all healthy

Scenario Timeline (15 min)

Min 0–5	All services normal
Min 5–7	Response times creep up
Min 7–8	**Model fires alert**
Min 9–12	Full incident develops
Min 12–15	Recovery phase

ML Model

Gradient Boosting Classifier with KPI + Log-Proxy Features

Model Configuration

Parameter	Value
Algorithm	GBC (scikit-learn)
Trees	100
Max Depth	3
Learning Rate	0.1
Horizons	5, 10, 15 min
Training Data	2.4M timesteps

Alert Calibration

Thresholds tuned per service so that at most **1 false alarm per 24 hours**. Frozen after training for consistent deployment.

8 Engineered Features

Feature	Type	Import.
roll_mean	KPI	31.1%
error_rate	Log	26.6%
roll_max	KPI	19.8%
roll_std	KPI	12.9%
severity_change	Log	4.6%
first_diff	KPI	3.5%
roll_slope	KPI	1.0%
warn_rate	Log	0.4%

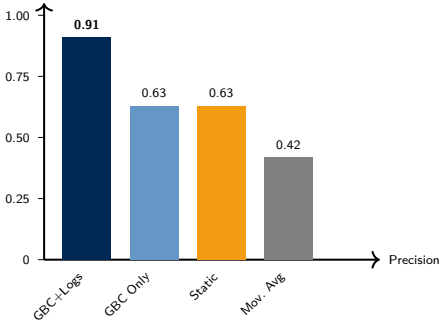
Key insight: Log-proxy features (red) contribute **31.6%** of total importance, multi-signal fusion is essential.

Model Performance

Best Result: GBC with KPI + Log Features at H=15

Performance Metrics

Metric	Value
Precision	**0.91**
Mean Lead Time	**$\sim$36 min**
False Alarm Rate	$\leq$ 1/day/service
Detection Rate	8.7%



KPI-Only vs KPI+Logs

Model	Prec.	Rec.
GBC KPI+Logs	**0.91**	**0.43**
GBC KPI Only	0.63	0.16
Static Baseline	0.63	0.47
Moving Avg.	0.42	0.02

Key Finding

The main operational benefit: **more trustworthy alerts with less alert fatigue**, not just earlier detection.

Implementation: Backend

13 Lambda Functions with Event-Driven Processing

Lambda Functions (from AWS)

Function	Trigger
simulator_lambda	EventBridge
feature_engineering	DynamoDB Stream
inference_lambda	DynamoDB Stream
services_lambda	API Gateway
kpi_lambda	API Gateway
alerts_lambda	API Gateway
incidents_lambda	API Gateway
analytics_lambda	API Gateway
threshold_lambda	API Gateway
control_simulation	API Gateway
get_metrics_lambda	API Gateway
get_logs_lambda	API Gateway
notification_lambda	API Gateway

Inference Lambda Details

- Deployed as **Docker container** on Lambda
- Loads 3 trained GBC models from S3
- **Model caching** avoids re-download
- Processes DynamoDB Stream events
- Creates alerts when score $>$ threshold
- Groups $\geq$2 related alerts into incidents

API Gateway Configuration

- HTTP API with **19 routes**
- All routes protected by **JWT Authorizer**
- CORS configured for frontend origin
- Rate limiting: 10K req/s burst

Security Architecture

Authentication, Authorization, and Encryption

Authentication (AWS Cognito)

- SRP protocol (password never sent in plaintext)
- Email verification required
- Password policy: min 8 chars, upper + lower + number
- JWT tokens: ID (1h), Refresh (30d)

Encryption

Scope	Method
DynamoDB at rest	AES-256 (AWS keys)
S3 at rest	AES-256 (SSE-S3)
API calls in transit	TLS 1.2+
Auth in transit	TLS 1.2+
AWS internal	TLS 1.2+

Authorization

- API Gateway JWT Authorizer on **all 16 routes**
- Frontend ProtectedRoute component
- 401 response → auto-redirect to /login

Access Controls

- Least-privilege IAM roles for Lambda
- No direct public access to DynamoDB
- S3: public read for frontend only
- No hardcoded secrets in source code

Verification

`curl .../services` → `{"message":"Unauthorized"}`

HTTP 401 without valid JWT token.

Live Demo

Watch the Model Detect an Incident in Real Time

Open Dashboard Demo

Security Audit & Testing

OWASP Top 10 Assessment + Integration Test Results

OWASP Top 10 Results

#	Vulnerability	Status
A01	Broken Access Control	Pass
A02	Cryptographic Failures	Pass
A03	Injection	Pass
A04	Insecure Design	Pass
A05	Security Misconfig.	Pass
A06	Vulnerable Components	Pass
A07	Auth Failures	Pass
A08	Data Integrity	Pass
A09	Security Logging	Pass
A10	SSRF	N/A

Score: 9/10 passed, 1 N/A. Overall risk: **Low**.

Integration Testing Summary

Category	Tests
Authentication (CLI)	5 passed
API Endpoints	16 tested
Simulation Lifecycle	Verified
Database Integrity	Verified
Production Deployment	Verified
Authentication (Browser)	4 planned
Frontend Integration	17 planned
Cross-Cutting (Theme, etc.)	7 planned
Total	**50+ cases**

Additional Checks

CORS, Rate Limiting, Input Validation, Error Handling, Secrets Management, DynamoDB TTL, all passed.

Challenges and Solutions

Key Obstacles Encountered During Implementation

Challenge 1: Lambda Package Size Limitation

Problem: Deploying the model with Python ML libraries exceeded AWS Lambda package size limits.
Solution: Used Docker container deployment to package the dependencies and model together, avoiding the space limitation.

Challenge 2: Heterogeneous Data Schema

Problem: 5 services produce completely different JSON structures.
Solution: Used DynamoDB's flexible schema, each service stores its own document shape. Feature Engineering extracts one primary metric per service for the model.

Conclusion and Future Work

What We Built and What Comes Next

What We Built

1. A **fully serverless** AIOps platform on AWS
2. **ML-powered** incident prediction with 0.91 precision
3. **Event-driven** pipeline: data → features → alerts (automatic)
4. **Interactive dashboard** with real-time visualization
5. **Comprehensive security**: Cognito auth, JWT, AES-256, OWASP audit

Key Numbers

13 Lambda Functions	6 DynamoDB Tables
19 API Routes (JWT)	5 Monitored Services
3 ML Models	50+ Test Cases
8 Engineered Features	9/10 OWASP Passed

Core Achievement

Predictive models don't just detect earlier, they produce **more trustworthy alerts** with **fewer false alarms**, leading to higher operator trust.

Future Work

- Integrate **real system logs** and production traffic
- Add **multi-metric models** for each service
- Explore deeper models such as **LSTM** and **Transformer**
- Apply **hyperparameter tuning** to improve recall
- Add **cross-service correlation** for cascading failure detection

Thank you for your attention!

Questions?

Team: Akinbobola Akin · Alfarizy Alfarizy · Ishimwe Pacis Hanyurwimfura

Live Dashboard:

`http://cloud-project-dashboard1.s3-website-us-east-1.amazonaws.com/`

API Endpoint:

`p9fpx4nhh6.execute-api.ca-central-1.amazonaws.com`